

Sztringek Pythonban: a `str`

Jegyzet és feladatlap

Hogyan dolgozz ezzel a jegyzettel?

Ugyanúgy, mint a listás jegyzettel: olvass, próbáld ki, aztán oldd meg a feladatokat.

- **Próbáld ki** – gépeld be a kódot, és futtasd. Gépeld, ne másold: a kezed is tanul.
- **Tippelj** – előbb írd le a füzetedbe, szerinted mit ír ki a program, és csak utána futtasd.
- **Feladat** – ezt már te írod meg. A nehézséget csillagok jelzik: ★ = alap, ★★ = közepes, ★★★ = nehezebb.

Néhány tudnivaló, mielőtt belevágsz:

- Amit a társaid C++-ban `std::string`-nek hívnak, az Pythonban a `str`. A feladatok szinte mind ugyanazok, úgyhogy bátran beszéljétek meg őket.
- Néhány dolog Pythonban másképp működik, mint C++-ban. Ezeket a jegyzet külön jelzi.
- Jó hír: amit a listáknál megtanultál (indexelés, bejárás, megszámlálás, keresés), az itt is működik.

1. Szöveg egy változóban

A sztring (`str`) szöveget tárol: betűket, számjegyeket, szóközöket, írásjeleket.

Próbáld ki

```
nev = "Anna"

print("Szia,", nev)
```

A szöveget idézőjelek közé írjuk. Pythonban mindegy, hogy dupla (`"Anna"`) vagy szimpla (`'Anna'`) idézőjelet használsz, csak a szöveg két végén ugyanaz legyen. Írd át a nevet a sajátodra, és futtasd újra.

Próbáld ki - összefűzés

```
elso = "alma"
masodik = "fa"
egyutt = elso + masodik

print(egyutt)
```

A `+` jel két sztringet egymás mögé ragaszt.

Tippelj

```
a = "12"
b = "34"
print(a + b)

szo = "ha"
szo += "ha"
szo += "!"
print(szo)

print("ha" * 3)
print("-" * 20)
```

Az első kiírásnál 46-ot vártál? Fogalmazd meg a füzetedben, miért nem azt kaptad. Mit csinál a `*` jel egy sztringgel?

Kísérlet - szöveg és szám

```
kor = 17
print("Eletkor: " + kor)
```

Olvasd el a hibaüzenet utolsó sorát. Mit nem enged a Python?

Figyelem! A `+` jellel csak szöveget lehet szöveghez fűzni. Ha számot akarsz hozzáfűzni, előbb alakítsd szöveggé: `"Eletkor: " + str(kor)`. A `print("Eletkor:", kor)` alak vesszővel ezt megoldja helyetted.

Próbáld ki - a szöveg hossza

```
szo = "informatika"
print(len(szo))
```

Ugyanaz a `len()`, mint a listáknál.

Tippelj - mi számít bele a hosszba?

Mekkora a hossza ennek a három sztringnek? Írd le a tippedet, aztán ellenőrizd.

```
a = ""
b = " "
c = "Kiss Anna"
```

Feladatok

1.1. feladat ★☆☆ - Köszönés

Tárold el a keresztnedet egy változóban. Írasd ki vele ezt a sort: `Szia, Anna! Udv a szakkoron.` Figyelj arra, hogy a felkiáltójel előtt ne legyen szóköz.

1.2. feladat ★☆☆ - Teljes név

Legyen két változód: `vezeteknev` és `keresztnev`. Fűzd össze őket egy harmadik, `teljes` nevű változóba úgy, hogy szóköz legyen közöttük. Írd ki a `teljes` értékét.

1.3. feladat ★☆☆ - Milyen hosszú?

Írd ki a teljes név hosszát. Számold meg kézzel is: stimmel? Mit számolt bele a program a betűkön kívül?

1.4. feladat ★☆☆ - Nevetés

Indulj egy üres sztringből. Ciklussal fűzd hozzá ötször a `"ha"` szöveget, majd írd ki az eredményt és a hosszát. Utána oldd meg ciklus nélkül, egyetlen sorban is.

1.5. feladat ★☆☆ - Aláhúzás

Írd ki az `informatika` szót, alá pedig egy új sorba pontosan annyi kötőjelet, ahány betűből a szó áll. Írd át a szót másra: a kötőjelek száma magától igazodjon.

2. Beolvasás: szó vagy sor?

Az `input()` mindig egy egész sort olvas be, és az eredménye mindig szöveg.

Próbáld ki

```
nev = input("Mi a neved? ")  
  
print("Szia, " + nev + "!")
```

Futtasd kétszer. Először csak a keresztnévedet írd be, másodszor a teljes nevedet. Megkaptad mindkét szót?

Kérdezd meg a C++-os társaidat, náluk mi történt ugyanennél a kísérletnél!

Kísérlet - az `input()` mindig szöveget ad

```
kor = input("Hány éves vagy? ")  
  
print(kor * 2)  
print(kor + 1)
```

Az első kiírás meglepő, a második hibát ad. Miért? Gondolj az 1. fejezetre.

Most már érted, miért írtuk eddig mindig így: `kor = int(input())`. Az `int()` alakítja a beolvasott szöveget számmá.

Próbáld ki - egy sor szavakra bontva

Ha egy sorban több szó van, a `split()` szétszedi őket:

```
sor = input("Vezeteknev es keresztnév: ")  
szavak = sor.split()  
  
print(szavak)  
print(szavak[1], szavak[0])
```

A `split()` eredménye egy lista, amelyben a szavak állnak. A listákat pedig már ismered.

Próbáld ki - két szó két változóba

```
vezeteknev, keresztnév = input().split()  
  
print(keresztnév, vezeteknev)
```

Írd be: `Kiss Anna`. Aztán futtasd újra három szóval: `Kiss Anna Maria`. Mit mond a hibaüzenet?

Próbáld ki - szavak listába

```
szavak = []  
  
for i in range(3):  
    szo = input()  
    szavak.append(szo)  
  
for i in range(len(szavak)):  
    print(i + 1, "- szo:", szavak[i])
```

Itt minden szót külön sorba kell írni, mert minden `input()` egy teljes sort olvas be.

Feladatok

2.1. feladat ★☆☆ - Bemutatkozás

Kérd be a felhasználó keresztnévét és életkorát (két külön `input()`). Írd ki így:

Anna 17 éves, jövőre 18 lesz.

2.2. feladat ★☆☆ - Fordított név

Olvasd be egy sorból a vezeték- és a keresztnévet (két szó). Írd ki őket fordított sorrendben, vesszővel:

Anna, Kiss.

2.3. feladat ★☆☆ - Mondat hossza

Olvass be egy egész mondatot, és írd ki, hány karakterből áll.

2.4. feladat ★☆☆ - Kor és név

A felhasználó egy sorba írja a keresztnévét és az életkorát: Anna 17. Írd ki, hány éves lesz jövőre. **Tipp:** a `split()` után az életkor még szöveg.

2.5. feladat ★☆☆ - Bevásárlólista

Kérdezd meg, hány tételből áll a lista. Olvass be ennyi tételt (mindegyiket külön sorból) egy listába, majd írd ki őket sorszámozva. Egy tétel több szóból is állhat, például friss kenyér.

2.6. feladat ★☆☆ - Lista egy sorban

Most a felhasználó az egész bevásárlólistát egyetlen sorba írja, szóközökkel elválasztva: tej kenyér alma. Írd ki a tételeket sorszámozva, és azt is, hány tétel van.

3. A sztring karakterei

A sztring karakterek sorozata. Az elemeit ugyanúgy indexeled és járod be, mint egy listáét.

Próbáld ki

```
szo = "alma"

print(szo[0])
print(szo[3])
```

Sorszám (index)	0	1	2	3
Karakter	a	l	m	a

Tippelj

Mit ír ki ez a két sor?

```
szo = "informatika"
print(szo[len(szo) - 1])
print(szo[-1])
```

Kísérlet - módosítható-e egy betű?

A listák elemeit át lehetett írni. Próbáld ki ugyanezt egy sztringgel:

```
szo = "fal"
szo[0] = "h"
print(szo)
```

Olvasd el a hibaüzenet utolsó sorát.

Figyelem! Pythonban a sztring **nem módosítható**: egy betűjét nem lehet átírni. Ha más szöveg kell, új sztringet kell létrehozni. Erről szól a 6. fejezet. (C++-ban a sztring betűi átírhatók – ez az egyik nagy különbség.)

Egy karakter Pythonban

Pythonban nincs külön karaktertípus. Egy karakter egyszerűen egy egy hosszúságú sztring:

```
szo = "alma"
elso = szo[0]

print(elso, len(elso))
```

Próbáld ki - bejárás

A sztringet mindkét módon bejárhatod, ahogy a listát:

```
szo = "informatika"

for i in range(len(szo)):
    print(szo[i])
```

```
for betu in szo:
    print(betu)
```

Próbáld ki - megszámlálás

Hány **a** betű van a szóban?

```
szo = "informatika"

db = 0
for betu in szo:
    if betu == "a":
        db = db + 1

print(db)
```

Tipp: Ellenőrzésre: a `szo.count("a")` megszámolja az előfordulásokat, az `"a" in szo` pedig megmondja, van-e ilyen betű a szóban. A feladatokban azért te írd meg a ciklust.

Feladatok

3.1. feladat ★☆☆ - Első és utolsó

Olvass be egy szót, és írd ki az első és az utolsó betűjét.

3.2. feladat ★☆☆ - Betűnként

Írd ki a beolvasott szó betűit egy sorba, szóközzel elválasztva: `a l m a`.

3.3. feladat ★☆☆ - Visszafelé

Írd ki a beolvasott szót visszafelé: az `alma` szóból `amla` legyen. **Tipp:** a `range()` három számmal, ahogy a listáknál.

3.4. feladat ★☆☆ - Hány e betű?

Olvass be egy egész mondatot. Számold meg, hány **e** betű van benne.

3.5. feladat ★★☆☆ - Szóközök

Számold meg, hány szóköz van a beolvasott mondatban. Hány szóból áll a mondat, ha a szavak között mindig pontosan egy szóköz van?

3.6. feladat ★★☆☆ - Magánhangzók

Számold meg, hány magánhangzó (a, e, i, o, u) van a beolvasott mondatban.

3.7. feladat ★★☆☆ - Aláhúzás

Írd ki a beolvasott mondatot úgy, hogy minden szóköz helyén aláhúzásjel (_) áll: `a_kutya_ugat`. **Tipp:** írd ki betűnként, `end=""` beállítással.

3.8. feladat ★★☆☆ - Van-e benne?

Döntsd el ciklussal, van-e `z` betű a beolvasott szóban. **Tipp:** logikai változó, ahogy a listás keresésnél.

4. Karakterek vizsgálata és átalakítása

A számítógép minden karaktert egy számként, a kódjával tárol. Ezt kihasználva vizsgálhatod és alakíthatod a betűket.

Próbáld ki - a karakter kódja

Az `ord()` megadja egy karakter kódját:

```
c = "a"

print(c, "kódja:", ord(c))
```

Próbáld ki a táblázat többi karakterével is, és írd be, amit kapsz:

Karakter	"a"	"b"	"z"	"A"	"Z"	"0"	"9"	" "
Kódja	97							

Mit veszel észre? Hogyan követik egymást a betűk? Melyik a kisebb: a kisbetűk vagy a nagybetűk kódja?

Tippelj - kódból karakter

A `chr()` az `ord()` fordítottja: a kódból csinál karaktert.

```
print(chr(97))
print(chr(ord("a") + 1))
print(chr(ord("A") + 25))
```

Próbáld ki - milyen karakter ez?

Mivel a betűk kódjai sorban követik egymást, összehasonlítással eldöntheted, mi micsoda:

```
c = "k"

if "a" <= c <= "z":
    print("kisbetu")
```

Erre kész metódusok is vannak:

Metódus	Mikor igaz?
<code>c.isalpha()</code>	ha <code>c</code> betű
<code>c.isdigit()</code>	ha <code>c</code> számjegy
<code>c.islower()</code>	ha <code>c</code> kisbetű
<code>c.isupper()</code>	ha <code>c</code> nagybetű
<code>c.isspace()</code>	ha <code>c</code> szóköz, tabulátor vagy sortörés

Kísérlet - kisbetűből nagybetű

Az `upper()` és a `lower()` egész szövegre is működik. Tippelj: mit ír ki a **második** `print`?

```
szo = "informatika"

print(szo.upper())
print(szo)
```

A `szo` nem változott meg! Az `upper()` egy új, nagybetűs sztringet ad vissza, az eredetihez nem nyúl. Ha a változást meg akarod tartani, tedd vissza a változóba:

```
szo = szo.upper()
print(szo)
```

Figyelem! A sztring metódusai soha nem módosítják az eredeti sztringet, hanem új sztringet adnak vissza. Ha az eredményt nem teszed el egy változóba, elvesz.

Próbáld ki - számjegyből szám

```
c = "7"
ertek = int(c)

print(ertek + 1)
```

Mit ír ki a program, ha az `int()` elmarad, és a `c + 1` értékét próbálsd kiírni?

Kísérlet - ékezetes betűk

Hány betűből áll a `korte` és a `körte` szó? Mit mond a program?

```
a = "korte"
b = "körte"

print(len(a), len(b))
print("a" <= "ö" <= "z", "ö".isalpha())
```

Tipp: Pythonban az ékezetes betű is egyetlen karakter. Az `"a" <= c <= "z"` vizsgálat viszont csak az angol ábécé betűire igaz, az ékezetesekre nem. Az `isalpha()` rájuk is igazat ad.

Feladatok

4.1. feladat ★☆☆ - Kódtábla

Írd ki az angol ábécé kisbetűit a kódjukkal együtt, soronként egyet: **a 97. Tipp:** a `range()` menjen végig a kódokon, az `"a"` kódjától a `"z"` kódjáig.

4.2. feladat ★☆☆ - Csupa nagybetű

Olvass be egy mondatot, és írd ki csupa nagybetűvel.

4.3. feladat ★☆☆ - Számjegyek száma

Olvass be egy sort, amelyben betűk és számok vegyesen vannak (például `Petofi utca 12, 3. emelet 14`). Hány számjegy van benne?

4.4. feladat ★☆☆ - Számjegyek összege

Olvass be egy számot sztringként (például `2026`), vagyis `int()` nélkül. Add össze a számjegyeit. A példában az eredmény 10.

4.5. feladat ★☆☆ - Nagy kezdőbetű

Olvass be egy nevet, akárhogyan is van írva (`aNNa`). Írd ki helyesen: az első betű nagy, a többi kicsi (`Anna`). **Tipp:** építsd fel a kiírást betűnként, az index alapján döntve.

4.6. feladat ★☆☆ - Karakterstatisztika

Olvass be egy sort. Számold meg külön a kisbetűket, a nagybetűket, a számjegyeket és az összes többi karaktert.

4.7. feladat ★★★ - Jelszóellenőr

Egy jelszó akkor erős, ha legalább 8 karakter hosszú, és van benne kisbetű, nagybetű és számjegy is. Olvass be egy jelszót, és írd ki, erős-e. Ha nem, sorold fel, melyik feltétel nem teljesül.

5. Részsztring és keresés

A szeletelés kivág egy darabot a szövegből. A `find` megmondja, hol van benne egy betű vagy egy szövegrész.

Próbáld ki - szeletelés

```
szo = "informatika"

print(szo[0:4])
print(szo[2:7])
print(szo[6:])
```

A táblázat segít kitalálni, mit jelent a két szám:

Sorszám (index)	0	1	2	3	4	5	6	7	8	9	10
Karakter	i	n	f	o	r	m	a	t	i	k	a

A `szo[a:b]` az `a` indextől a `b` index **előttig** tart – pontosan úgy, ahogy a `range(a, b)`. Ha az első számot elhagyod, az elejétől indul; ha a másodikat, a végéig tart.

Figyelem! A C++-os társaid `substr`-je másképp számol: ott a második szám darabszám. Pythonban a második szám a vég indexe, amely már nincs benne a szeletben.

Tippelj - szelet-totó

Töltsd ki a középső oszlopot fejből, a fenti táblázat alapján. Utána ellenőrizd géppel. A **szo** értéke végig "informatika".

Kifejezés	A tippem	A Python szerint
szo[0:2]		
szo[4:7]		
szo[8:]		
szo[:5]		
szo[-3:]		
szo[3:100]		
szo[20:]		

Az utolsó két sor meglepő lehet. Emlékezz vissza: mit csinált a Python, amikor egy listát a végén túl indexeltél? És mit csinál most?

Próbáld ki - keresés

```
mondat = "a kutya ugat"

print(mondat.find("kutya"))
print(mondat.find("u"))
print(mondat.find("ugat"))
```

A **find** annak az indexét adja meg, ahol a keresett szöveg **először** kezdődik.

Kísérlet - ha nincs találat

```
print(mondat.find("macska"))
```

Mit kaptál? Miért jó jelzés ez arra, hogy nincs találat? (Gondolj a listás keresésre.)

Ha csak az érdekel, benne van-e, az **in** a legegyszerűbb:

```
if "macska" in mondat:
    print("Van benne.")
else:
    print("Nincs benne.")
```

Próbáld ki - keresés egy adott helytől

A **find** második száma megmondja, honnan induljon a keresés:

```
mondat = "a kutya ugat"

print(mondat.find("a"))
print(mondat.find("a", 1))
print(mondat.find("a", 7))
```

Tippelj, aztán futtasd. Hogyan találod meg egymás után az összes **a** betűt?

Feladatok

5.1. feladat ★☆☆ - Eleje és vége

Olvass be egy legalább hatbetűs szót. Írd ki az első három és az utolsó három karakterét.

5.2. feladat ★☆☆ - Monogram

Olvasd be a vezeté- és a keresztnévet. Írd ki a monogramot pontokkal: **K. A.**

5.3. feladat ★☆☆ - E-mail-cím

Olvass be egy e-mail-címet (például **anna.kiss@iskola.hu**). Írd ki külön a @ előtti és a @ utáni részt.

Tipp: keresd meg a @ helyét, és szeletelj.

5.4. feladat ★☆☆ - Kiterjesztés

Olvass be egy fájlnevet (például **dolgozat.docx**). Írd ki a kiterjesztését, vagyis a pont utáni részt. Ha nincs benne pont, ezt írd ki: **Nincs kiterjesztes.**

5.5. feladat ★☆☆ - Dátum

Olvass be egy dátumot ebben a formában: **2026.10.04**. Írd ki külön sorba az évet, a hónapot és a napot. Most szeleteléssel oldd meg.

5.6. feladat ★☆☆ - Benne van-e?

Olvass be egy mondatot, majd egy keresett szót. Írd ki, szerepel-e a szó a mondatban, és ha igen, hányadik indexen kezdődik.

5.7. feladat ★★ - Hányszor?

Számold meg, hányszor fordul elő a keresett szövegrész a mondatban. **Tipp:** ciklusban keress a **find**-dal, mindig az előző találat utáni helytől indulva, amíg van találat. Az eredményt ellenőrizd a **mondat.count()** metódussal.

6. Sztring átalakítása és építése

Mivel a sztring nem módosítható, minden átalakítás egy új sztringet hoz létre: szeleteléssel, kész metódussal vagy karakterenkénti építéssel.

Tippelj - törlés, beszúrás, csere szeleteléssel

Mindhárom kiírás előtt tippelj. Az index-táblázat az előző fejezetben van.

```
s = "informatika"

s = s[2:]
print(s)

s = "in" + s
print(s)

s = "re" + s[2:]
print(s)
```

Próbáld ki - csere és levágás

```
s = "informatika"

print(s.replace("a", "e"))
print(s.replace("matika", ""))
print(s)

t = "  alma  "
print(len(t), len(t.strip()))
```

Metódus	Mit ad vissza?
<code>s.replace(mit, mire)</code>	új sztring, amelyben a mit minden előfordulása ki van cserélve
<code>s.strip()</code>	új sztring, amelynek az elejétől és a végétől le vannak vágva a szóközök és a sortörések

Próbáld ki - új sztring építése

Így tünteted el a szóközöket egy mondatból:

```
mondat = "a kutya ugat"
uj = ""

for betu in mondat:
    if betu != " ":
        uj += betu

print(uj)
```

Ismerős? Ez a kiválogatás a listás jegyzetből. Az `append` helyett itt a `+=` fűzi az új karaktert a sztring végére.

Tippelj

Ez a ciklus csak egy apróságban tér el az előzőtől. Mit ír ki?

```
szo = "alma"
uj = ""

for betu in szo:
    uj = betu + uj

print(uj)
```

Mi a különbség az `uj += betu` és az `uj = betu + uj` között?

Tipp: Megfordításra van rövid út is: a `szo[::-1]` szelet. Ellenőrzésre használd; a feladatokban a ciklust írd meg.

Feladatok

6.1. feladat ★☆☆ - Egyetlen csere

Az `informatika` szóból szeleteléssel és összefűzéssel csinálj `matematika` szót, a `replace` használata nélkül.

6.2. feladat ★☆☆ - Szóközök nélkül

Olvass be egy mondatot, és írd ki szóközök nélkül. Oldd meg ciklussal, aztán egyetlen `replace` hívással is.

6.3. feladat ★☆☆ - Megfordítás

Olvass be egy szót. Állítsd elő a megfordítását egy új sztringben, és írd ki mindkettőt.

6.4. feladat ★☆☆ - Palindrom

A palindrom visszafelé olvasva is ugyanaz, például `gorog` vagy `kajak`. Döntsd el a beolvasott szóról, hogy palindrom-e. **Tipp:** két sztringet az `==` jellel hasonlíthatsz össze.

6.5. feladat ★☆☆ - Magánhangzók nélkül

Írd ki a beolvasott szót a magánhangzói nélkül: az `informatika` szóból `nfrmtk` lesz.

6.6. feladat ★★☆☆ - Betűcsere

Cseréld ki a beolvasott mondatban minden **a** betűt **e** betűre. Írd meg ciklussal, új sztringet építve.

6.7. feladat ★★☆☆ - Dadogás

Duplázd meg a beolvasott szó minden betűjét: az **alma** szóból **aallmmaa** lesz.

6.8. feladat ★★☆☆ - Cenzúra

Olvass be egy mondatot és egy tiltott szót. A tiltott szó **első** előfordulását cseréld ki ugyanannyi csillagra, ahány betűből áll. Ha a szó nincs a mondatban, a mondat maradjon változatlan. **Tipp:** **find** és **szeletelés**.

7. Összehasonlítás és rendezés

Két sztringet ugyanazokkal a jelekkel hasonlítsz össze, mint két számot. Az a „kisebb”, amelyik a szótárban előbb áll.

Próbáld ki

```
a = input("Elso szo: ")
b = input("Masodik szo: ")

if a == b:
    print("Egyformak.")
elif a < b:
    print(a, "van elobb.")
else:
    print(b, "van elobb.")
```

Tippelj - melyik van előbb?

Töltsd ki a tippetet, aztán ellenőrizd a fenti programmal.

a	b	Tippem: melyik van előbb?	A Python szerint
alma	korte		
alma	almafa		
abc	abd		
Zebra	alma		
alma	Alma		
10	9		

Az utolsó három sor meglepő lehet. A magyarázat a 4. fejezetben van: a program karakterenként, a **kódok** alapján hasonlít. A nagybetűk kódja kisebb, mint a kisbetűké. A **10** és a **9** pedig itt szöveg, nem szám: az **1** kódja kisebb, mint a **9** kódja.

Tipp: Ha a kis- és nagybetűk különbsége nem számít, alakítsd mindkét szót kisbetűssé az összehasonlítás előtt.

Próbáld ki - rendezés

A `sort()` növekvő sorrendbe rendezi a listát:

```
nevek = ["Dani", "Anna", "Csilla", "Bence"]

nevek.sort()

for nev in nevek:
    print(nev)
```

Próbáld ki számokat tároló listával is. Működik?

Figyelem! A `sort()` kivétel az eddigi szabály alól: magát a listát rendezi át, és nem ad vissza semmit. A lista ugyanis módosítható, a sztring nem.

Feladatok

7.1. feladat ★☆☆ - Jelszó kétszer

Kérd be kétszer ugyanazt a jelszót. Írd ki, hogy a két beírás egyezik-e.

7.2. feladat ★☆☆ - Ábécérendben

Olvass be két szót, és írd ki őket ábécérendben.

7.3. feladat ★★☆☆ - Kisbetű, nagybetű

Olvass be két szót. Döntsd el, hogy egyformák-e, ha a kis- és nagybetűk különbségét nem vesszük figyelembe (`Alma` és `alma` egyformának számít).

7.4. feladat ★★☆☆ - Névsor

Kérdezd meg, hány név lesz, olvasd be őket egy listába, majd írd ki ábécérendben.

7.5. feladat ★★☆☆ - Leghosszabb szó

Olvass be öt szót egy listába. Írd ki a leghosszabbat. **Tipp:** maximumkiválasztás, csak most a szavak hosszát hasonlítod össze.

7.6. feladat ★★☆☆ - Rendezés nélkül

Olvass be öt szót, és írd ki azt, amelyik az ábécében a legelső. A `sort()` és a `min()` most nem használható.

8. Szám és szöveg között

Az `"123"` szöveg és az `123` szám nem ugyanaz. Az `int()` szövegből számot, az `str()` számból szöveget csinál.

Tippelj

```
a = "12"
b = "34"

print(a + b)
print(int(a) + int(b))
```

Függvény	Mit csinál?
<code>int(s)</code>	szövegből egész szám
<code>float(s)</code>	szövegből tizedes tört (tizedesponnttal)
<code>str(x)</code>	számból szöveg

Kísérlet - mit bír el az int()?

Próbáld ki egyesével, és írd be az eredményt. Ha a program hibával leáll, azt is jegyezd fel.

Kifejezés	Eredmény
<code>int("42")</code>	
<code>int(" 42 ")</code>	
<code>int("42abc")</code>	
<code>int("3.99")</code>	
<code>float("3.99")</code>	
<code>int(float("3.99"))</code>	

Figyelem! Az `int()` hibával leállítja a programot, ha a szövegben a számon kívül más is van. (A C++-os `std::stoi` engedékenyebb – kérdezd meg a társaidat!)

Próbáld ki - számból szöveg

```
ev = 2026
s = str(ev)

print(len(s))
print(s[0])
print("Az év: " + s)
```

Szöveggént a számnak hossza és karakterei vannak.

Próbáld ki - formázott szöveg (f-sztring)

Az f-sztringbe kapcsos zárójelek között változókat és számításokat írhatsz. A kezdő idézőjel elé egy `f` betű kerül:

```
nev = "Anna"
kor = 17
atlag = 4.6666

print(f"{nev} {kor} éves, jövőre {kor + 1} lesz.")
print(f"Átlaga: {atlag:.2f}")
```

A `:.2f` azt kéri, hogy a szám két tizedesjeggyel jelenjen meg. Próbáld ki `:.1f` és `:.0f` beállítással is.

Feladatok

8.1. feladat ★☆☆ - Összeadó

Olvass be két számot **sztringként**. Írd ki az összefűzésüket és az összegüket is.

8.2. feladat ★☆☆ - Hány jegyű?

Olvass be egy egész számot `int()` segítségével. Írd ki, hány számjegyből áll.

8.3. feladat ★☆☆ - Születési év

Olvass be egy születési dátumot (2009.05.17). Írd ki, hány éves lesz az illető 2026 végén. **Tipp:** vágd ki az évet, és alakítsd számmá.

8.4. feladat ★☆☆ - Óra és perc

Olvass be egy időpontot `ora:perc` formában (07:45). Írd ki, hány perc telt el éjfél óta. A példában 465.

8.5. feladat ★★☆☆ - Tükörszám

Olvass be egy egész számot `int()` segítségével. Döntsd el, hogy visszafelé olvasva is ugyanaz-e (például 12321).

8.6. feladat ★★☆☆ - Időtartam

Olvass be két időpontot `ora:perc` formában, ugyanarról a napról. Írd ki f-sztringgel, mennyi idő telt el közöttük, így: `2 ora 35 perc`.

9. Sor feldarabolása

A feladatokban az adatok gyakran soronként érkeznek, és egy sorban több adat áll. A `split()` darabolja fel a sort, az eredménye egy lista.

Próbáld ki - sor szavakra bontása

```
sor = input()
szavak = sor.split()

for szo in szavak:
    print(szo)

print("Szavak száma:", len(szavak))
```

Írj be olyan mondatot is, amelyben két szó között több szóköz van. Zavarja ez a programot?

Innentől a szavakkal mindent megtehetsz, amit egy lista elemeivel.

Próbáld ki - vegyes adatok egy sorban

A `split()` mindig szövegdarabokat ad. A számokat neked kell átalakítanod:

```
sor = "Anna 17 4.5"
reszek = sor.split()

nev = reszek[0]
kor = int(reszek[1])
atlag = float(reszek[2])

print(nev, "jovore", kor + 1, "eves lesz.")
```

Mi történik, ha az `int()` elmarad, és a `reszek[1] + 1` értékét próbálod kiszámolni?

Próbáld ki - más elválasztójel

Ha az adatokat nem szóköz, hanem például pontosvessző választja el, add meg a `split()`-nek:

```
sor = "Kiss Anna;17;4.5"
reszek = sor.split(";")

nev = reszek[0]
kor = int(reszek[1])
atlag = float(reszek[2])

print(f"{nev} ({kor}) atlaga: {atlag}")
```

Miért nem lett volna jó itt a sima `split()`?

Kísérlet - split() és split(" ")

Tippelj, mielőtt futtatod. Figyeld meg, hány elemű listákat kapsz!

```
print("a b".split())
print("a b".split(" "))
print("a;;b".split(";"))
```

Figyelem! Az üres zárójeles `split()` bármennyi szóközt egyetlen elválasztónak tekint. Ha megadod az elválasztójelet, akkor minden egyes jelnél vág, és üres darabok is keletkezhetnek.

Próbáld ki - számok egy sorból, és vissza

```
sor = "3 8 12 5"

szamok = []
for resz in sor.split():
    szamok.append(int(resz))

print(szamok)
```

A `join()` a `split()` fordítottja: egy lista szövegeit fűzi össze, a megadott elválasztóval.

```
szavak = ["a", "kutya", "ugat"]

print(" ".join(szavak))
print("-".join(szavak))
```

Feladatok

9.1. feladat ★☆☆ - Hány szó?

Olvass be egy mondatot, és írd ki, hány szóból áll. Próbáld ki úgy is, hogy két szó közé több szóközt írsz. Hasonlítsd össze a 3.5. feladat megoldásával: melyik számol jól?

9.2. feladat ★☆☆ - Szavak és hosszuk

Írd ki a beolvasott mondat szavait külön sorokba, mindegyik mellé a hosszát: `kutya 5`.

9.3. feladat ★☆☆ - Leghosszabb szó a mondatban

Írd ki a beolvasott mondat leghosszabb szavát.

9.4. feladat ★☆☆ - Fordított szórend

Írd ki a mondat szavait fordított sorrendben: `a kutya ugat` helyett `ugat kutya a`.

9.5. feladat ★☆☆ - Számok egy sorban

Olvass be egy sort, amelyben egész számok állnak szóközzel elválasztva (`3 8 12 5`). Nem tudod előre, hány szám lesz. Írd ki az összegüket és az átlagukat.

9.6. feladat ★☆☆ - Hosszú monogram

Olvass be egy teljes nevet, amely akárhány szóból állhat (`Kiss Anna Maria`). Írd ki a monogramját: `KAM`.

9.7. feladat ★☆☆ - Adatsor

Olvass be egy sort ebben a formában: `Kiss Anna;17;4.5`. Írd ki, hány éves lesz a tanuló jövőre, és hogy az átlaga eléri-e a 4.0-t.

9.8. feladat ★★ - Pontverseny

Előbb olvasd be, hány versenyző van. Utána minden versenyzőről egy sor érkezik `nev;pontszam` formában (`Nagy Bence;87`). Írd ki a legtöbb pontot elérő versenyző nevét és az átlagpontszámot, két tizedesjegyre.

10. Kihívások

Ezekhez nincs új tudnivaló, csak össze kell raknod, amit eddig tanultál. Válaszd azt, amelyik megtetszik.

10.1. feladat ★★☆☆ - Mondatpalindrom

Az *Indul a gorog aludni* mondat visszafelé is ugyanaz, ha a szóközöket és a kis- és nagybetűk különbségét nem nézzük. Írj programot, amely egy beolvasott mondatról ezt eldönti.

10.2. feladat ★★☆☆ - Nagy kezdőbetűk

Olvass be egy kisbetűvel írt teljes nevet (*kiss anna maria*). Írd ki úgy, hogy minden szó nagybetűvel kezdődik.

10.3. feladat ★★★★★ - Caesar-rejtjel

Olvass be egy kisbetűs szót. Told el minden betűjét hárommal az ábécében: *a* helyett *d*, *b* helyett *e*. Az ábécé végén körbe kell fordulni: *x* helyett *a*, *y* helyett *b*, *z* helyett *c*. Írd meg a visszafejtést is! **Tipp:** `ord()` és `chr()`.

10.4. feladat ★★★★★ - Betűstatisztika

Olvass be egy kisbetűs mondatot. Számold meg, melyik betű hányszor fordul elő, és írd ki a leggyakoribbat. **Tipp:** használj 26 elemű számláló listát; az `ord(c) - ord("a")` megadja, hányadik betű a *c* az ábécében.

10.5. feladat ★★★★★ - Rendszám

Egy régi típusú rendszám így néz ki: *ABC-123*. Három nagybetű, kötőjel, három számjegy. Olvass be egy szöveget, és dönts el, megfelel-e ennek a formának.

10.6. feladat ★★★★★ - Anagramma

Két szó anagramma, ha ugyanazokból a betűkből állnak, csak más sorrendben (*roka* és *kora*). Olvass be két szót, és dönts el, anagrammák-e. **Tipp:** a `sorted(szo)` a szó betűit rendezett listában adja vissza.

10.7. feladat ★★★★★ - Tömörítés

Olvass be egy szót, amelyben sok az ismétlődő betű (*aaabccddddd*). Írd ki tömörítve: minden betű után az áll, hányszor ismétlődik egymás után (*a3b1c2d4*).

Összefoglaló táblázat (puska)

Művelet	Jelentés
<code>len(s)</code>	karakterek száma
<code>s[i]</code> és <code>s[-1]</code>	az <code>i</code> indexű, illetve az utolsó karakter
<code>s[a:b]</code>	részsztring az <code>a</code> indextől a <code>b</code> index előttig
<code>a + b</code> és <code>s * 3</code>	összefűzés, illetve ismétlés
<code>input()</code>	egy egész sor beolvasása (mindig szöveg)
<code>"x" in s</code>	igaz, ha az <code>x</code> szerepel a sztringben
<code>s.find(mit)</code>	az első előfordulás indexe, vagy -1
<code>s.count(mit)</code>	az előfordulások száma
<code>s.replace(mit, mire)</code>	cseré (új sztringet ad)
<code>s.upper()</code> és <code>s.lower()</code>	nagybetűs, illetve kisbetűs változat
<code>s.strip()</code>	szóközök és sortörés levágása a két végétől
<code>c.isdigit()</code> és <code>c.isalpha()</code>	számjegy-e, betű-e
<code>ord(c)</code> és <code>chr(kod)</code>	karakter kódja, illetve a kódhoz tartozó karakter
<code>int(s)</code> és <code>float(s)</code>	szövegből szám
<code>str(x)</code>	számból szöveg
<code>s.split()</code>	darabolás szavakra (listát ad)
<code>s.split(";")</code>	darabolás a megadott elválasztójelnél
<code>" ".join(lista)</code>	a lista szövegeinek összefűzése
<code>f"{nev}: {x:.2f}"</code>	formázott szöveg, két tizedesjeggyel
<code>a == b</code> és <code>a < b</code>	összehasonlítás a karakterkódok szerint

Figyelem! Három dolog, amit soha ne felejts el: a sztring nem módosítható, a metódusai mindig új sztringet adnak vissza; az `input()` és a `split()` mindig szöveget ad, a számot neked kell átalakítanod; a szelet végindexe már nincs benne a szeletben.